COMP102 Lecture 6: Binary Search Trees

Search, Ordering Invariants, Height, and AVL Trees

Jalal Maaouni
May 3, 2026

Plan

1 Learning Outcomes
2 From Queues to Priority Queues
3 Heap Property
4 Array Representation
5 Insertion and Removal
6 Python heapq
7 Priority Scheduling
8 Complexity and Limits

Learning Outcomes

Learning outcomes

By the end of this lecture, students should be able to:

- define the **priority queue** abstract data type;
- distinguish a normal queue from a priority queue;
- define the **min-heap** and **max-heap** properties;
- explain why a heap is **not** a binary search tree;
- represent a binary heap using a Python list;
- trace insertion using **sift up**;
- trace removal using **sift down**;
- use Python's `heapq` module to implement priority queues;
- model a small **priority scheduling** problem;
- analyze the time complexity of heap operations.

Last lecture: BSTs used an ordering invariant to search for a specific value. This lecture uses a weaker invariant to repeatedly extract the next most important value.

From Queues to Priority Queues

A normal queue is not always enough

A normal queue is not always enough

Normal queue

First in, first out.

The first task that arrives is handled first.

A normal queue is not always enough

Normal queue

First in, first out.

The first task that arrives is handled first.

Problem

Some tasks are more urgent than others.

A normal queue is not always enough

Normal queue

First in, first out.

The first task that arrives is handled first.

Problem

Some tasks are more urgent than others.

Task	Arrival	Priority
Print homework	1	3
System update	2	2
Emergency shutdown	3	10

A normal queue is not always enough

Normal queue

First in, first out.

The first task that arrives is handled first.

Problem

Some tasks are more urgent than others.

Task	Arrival	Priority
Print homework	1	3
System update	2	2
Emergency shutdown	3	10

FIFO answers: **who came first?** A priority queue answers: **who matters most right now?**

Priority queue intuition

Priority queue intuition

Queue

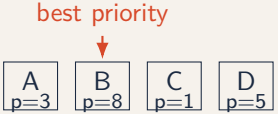
← A B C D

arrival order

Priority queue intuition

Queue

Priority queue



Priority queue intuition

Queue

← A B C D

arrival order

Priority queue

best priority

↓

A p=3 B p=8 C p=1 D p=5

A priority queue does not process items by position. It processes items by priority.

Priority queue as an ADT

Priority queue as an ADT

Abstract data type

A **priority queue** stores items with priorities and repeatedly gives access to the best-priority item.

Priority queue as an ADT

Abstract data type

A **priority queue** stores items with priorities and repeatedly gives access to the best-priority item.

Operation	Meaning
`push(item, priority)`	insert an item with priority
`peek()`	inspect the best item
`pop()`	remove and return the best item
`is_empty()`	check whether there are no items

Priority queue as an ADT

Abstract data type

A **priority queue** stores items with priorities and repeatedly gives access to the best-priority item.

Operation	Meaning
`push(item, priority)`	insert an item with priority
`peek()`	inspect the best item
`pop()`	remove and return the best item
`is_empty()`	check whether there are no items

The ADT says *what* behavior we want. We need a data structure to implement it efficiently.

Could we just use a list?

Implementation	Push	Pop best	Issue
Unsorted list	$O(1)$	$O(n)$	finding the best item is expensive
Sorted list	$O(n)$	$O(1)$	insertion is expensive
Binary heap	$O(\log n)$	$O(\log n)$	balanced compromise

Could we just use a list?

Implementation	Push	Pop best	Issue
Unsorted list	$O(1)$	$O(n)$	finding the best item is expensive
Sorted list	$O(n)$	$O(1)$	insertion is expensive
Binary heap	$O(\log n)$	$O(\log n)$	balanced compromise

A heap is not magic. It is a compromise: both insertion and removal stay efficient.

Heap Property

Two rules define a binary heap

Two rules define a binary heap

Rule 1: heap property

The parent is always at least as important as its children.

Two rules define a binary heap

Rule 1: heap property

The parent is always at least as important as its children.

Rule 2: shape property

The tree is **complete**: levels are filled from left to right.

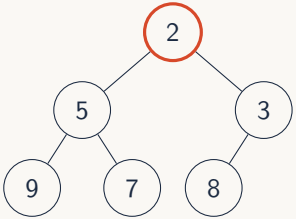
Two rules define a binary heap

Rule 1: heap property

The parent is always at least as important as its children.

Rule 2: shape property

The tree is **complete**: levels are filled from left to right.



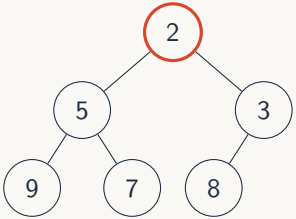
Two rules define a binary heap

Rule 1: heap property

The parent is always at least as important as its children.

Rule 2: shape property

The tree is **complete**: levels are filled from left to right.



The heap property gives access to the best item. The shape property keeps the height small.

The min-heap property

The min-heap property

Min-heap property

For every node:

$$\text{parent value} \leq \text{child values}$$

The min-heap property

Min-heap property

For every node:

$$\text{parent value} \leq \text{child values}$$

Remark 3.3. Consequence

The smallest value is always at the root.

The min-heap property

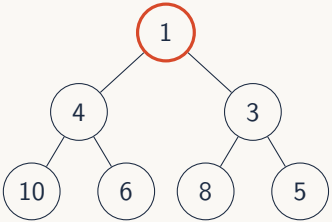
Min-heap property

For every node:

$$\text{parent value} \leq \text{child values}$$

Remark 3.4. Consequence

The smallest value is always at the root.

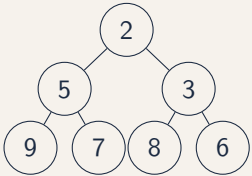


Min-heap and max-heap

Min-heap and max-heap

Min-heap

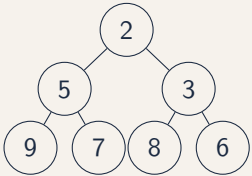
Parent $\leq$ children



Min-heap and max-heap

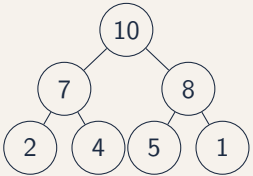
Min-heap

Parent $\leq$ children



Max-heap

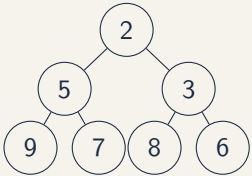
Parent $\geq$ children



Min-heap and max-heap

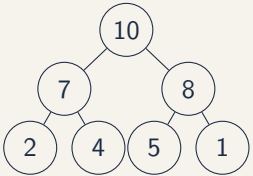
Min-heap

Parent $\leq$ children



Max-heap

Parent $\geq$ children



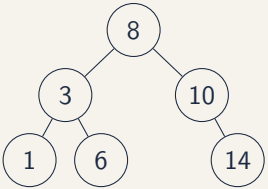
Same idea, opposite priority convention. In this lecture we mostly use min-heaps.

A heap is not a BST

A heap is not a BST

BST rule

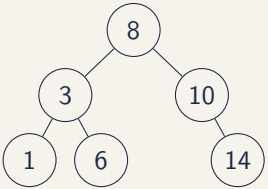
left subtree $<$ node $<$ right subtree



A heap is not a BST

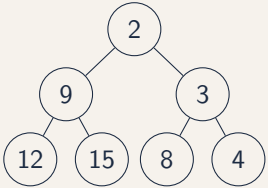
BST rule

left subtree $<$ node $<$ right subtree



Heap rule

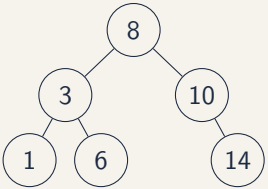
parent $\leq$ children



A heap is not a BST

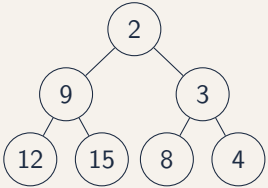
BST rule

left subtree $<$ node $<$ right subtree



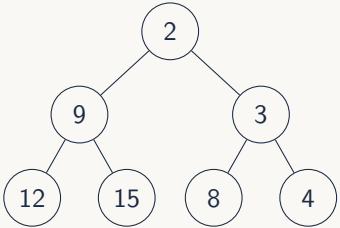
Heap rule

parent $\leq$ children

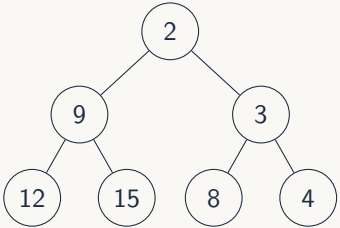


A heap only orders parents before children. It does not order the left subtree against the right subtree.

Valid heap but invalid BST



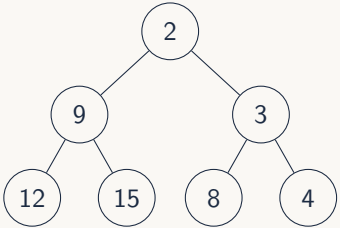
Valid heap but invalid BST



As a min-heap

Every parent is smaller than its children.

Valid heap but invalid BST



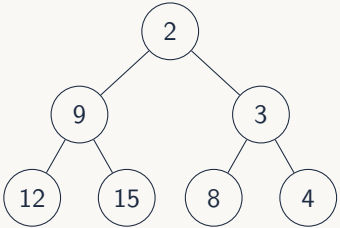
As a min-heap

Every parent is smaller than its children.

As a BST

The value 9 is in the left subtree of 2, but 9 is greater than 2.

Valid heap but invalid BST



As a min-heap

Every parent is smaller than its children.

As a BST

The value 9 is in the left subtree of 2, but 9 is greater than 2.

This tree is a valid min-heap and an invalid BST. The two invariants are different.

The complete-tree shape

The complete-tree shape

Complete binary tree

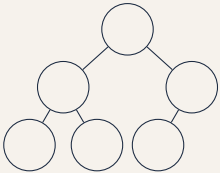
All levels are full except possibly the last, and the last level is filled from left to right.

The complete-tree shape

Complete binary tree

All levels are full except possibly the last, and the last level is filled from left to right.

Complete

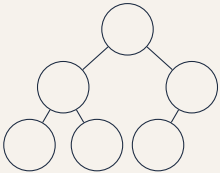


The complete-tree shape

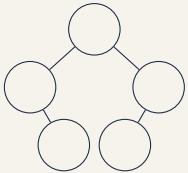
Complete binary tree

All levels are full except possibly the last, and the last level is filled from left to right.

Complete



Not complete

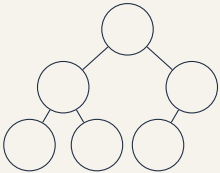


The complete-tree shape

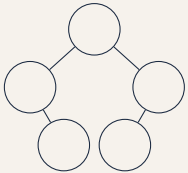
Complete binary tree

All levels are full except possibly the last, and the last level is filled from left to right.

Complete



Not complete



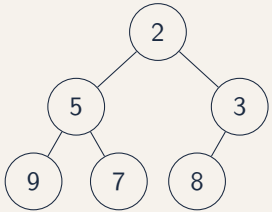
The shape rule prevents the heap from degenerating into a long chain.

Array Representation

A binary heap is stored as a list

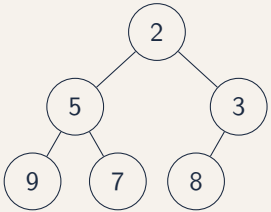
A binary heap is stored as a list

Tree view



A binary heap is stored as a list

Tree view



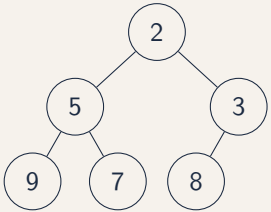
List view

2	5	3	9	7	8
0	1	2	3	4	5

`[2, 5, 3, 9, 7, 8]`

A binary heap is stored as a list

Tree view



List view

2	5	3	9	7	8
0	1	2	3	4	5

`[2, 5, 3, 9, 7, 8]`

The list stores the complete tree level by level, from left to right.

Index formulas

Index formulas

For a node at index `i`

Relation	Index
Left child	`2*i + 1`
Right child	`2*i + 2`
Parent	`(i - 1) // 2`

Index formulas

For a node at index `i`

Relation	Index
Left child	`2*i + 1`
Right child	`2*i + 2`
Parent	`(i - 1) // 2`

Example 4.3. Example

Index 1 has children:

$$2 \cdot 1 + 1 = 3, \qquad 2 \cdot 1 + 2 = 4$$

Index formulas

For a node at index `i`

Relation	Index
Left child	`2*i + 1`
Right child	`2*i + 2`
Parent	`(i - 1) // 2`

Example 4.4. Example

Index 1 has children:

$$2 \cdot 1 + 1 = 3, \qquad 2 \cdot 1 + 2 = 4$$

Remark 4.4. Important

This is why binary heaps usually do not need a `Node` class.

Index formulas

For a node at index `i`

Relation	Index
Left child	`2*i + 1`
Right child	`2*i + 2`
Parent	`(i - 1) // 2`

Example 4.5. Example

Index 1 has children:

$$2 \cdot 1 + 1 = 3, \qquad 2 \cdot 1 + 2 = 4$$

Remark 4.5. Important

This is why binary heaps usually do not need a `Node` class.

A complete-tree shape gives us a compact array representation.

Checking the heap property in a list

For a min-heap, every valid parent index `i` must satisfy:

$$\texttt{heap[i]} \leq \texttt{heap[2*i + 1]} \quad \text{and} \quad \texttt{heap[i]} \leq \texttt{heap[2*i + 2]}$$

Checking the heap property in a list

For a min-heap, every valid parent index `i` must satisfy:

$$\texttt{heap[i]} \leq \texttt{heap[2*i + 1]} \quad \text{and} \quad \texttt{heap[i]} \leq \texttt{heap[2*i + 2]}$$

Valid

```
[2, 5, 3, 9, 7, 8]
```

Checking the heap property in a list

For a min-heap, every valid parent index `i` must satisfy:

$$\texttt{heap[i]} \leq \texttt{heap[2*i + 1]} \quad \text{and} \quad \texttt{heap[i]} \leq \texttt{heap[2*i + 2]}$$

Valid

```
[2, 5, 3, 9, 7, 8]
```

Invalid

```
[2, 5, 1, 9, 7, 8]
```

Checking the heap property in a list

For a min-heap, every valid parent index `i` must satisfy:

$$\texttt{heap[i]} \leq \texttt{heap[2*i + 1]} \quad \text{and} \quad \texttt{heap[i]} \leq \texttt{heap[2*i + 2]}$$

Valid

```
[2, 5, 3, 9, 7, 8]
```

Invalid

```
[2, 5, 1, 9, 7, 8]
```

Only parent-child comparisons matter. The list does not need to be sorted.

Insertion and Removal

Insertion: append, then repair upward

Insertion: append, then repair upward

1. Put the new value at the next available position.

Insertion: append, then repair upward

1. Put the new value at the next available position.
2. Compare it with its parent.

Insertion: append, then repair upward

1. Put the new value at the next available position.
2. Compare it with its parent.
3. If the heap property is broken, swap with the parent.

Insertion: append, then repair upward

1. Put the new value at the next available position.
2. Compare it with its parent.
3. If the heap property is broken, swap with the parent.
4. Repeat until the heap property is restored.

Insertion: append, then repair upward

1. Put the new value at the next available position.
2. Compare it with its parent.
3. If the heap property is broken, swap with the parent.
4. Repeat until the heap property is restored.

Name of the repair
sift up

Insertion: append, then repair upward

1. Put the new value at the next available position.
2. Compare it with its parent.
3. If the heap property is broken, swap with the parent.
4. Repeat until the heap property is restored.

Name of the repair

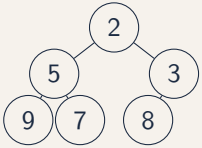
sift up

Insertion only repairs one path: from the new leaf toward the root.

Trace: inserting 1

Trace: inserting 1

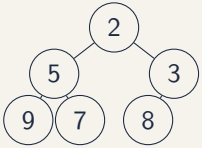
Start



[2,5,3,9,7,8]

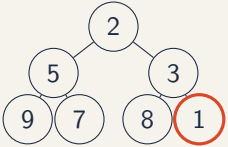
Trace: inserting 1

Start



[2,5,3,9,7,8]

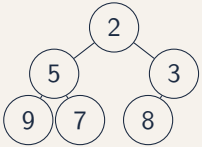
Append



[2,5,3,9,7,8,1]

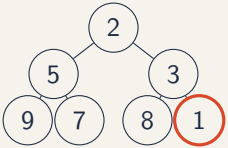
Trace: inserting 1

Start



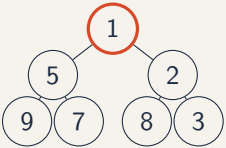
[2,5,3,9,7,8]

Append



[2,5,3,9,7,8,1]

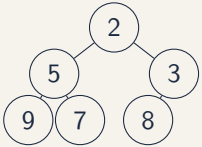
Sift up



[1,5,2,9,7,8,3]

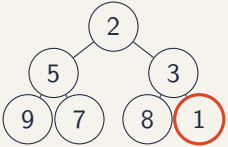
Trace: inserting 1

Start



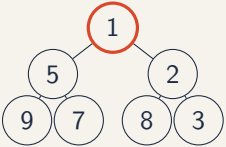
`[2,5,3,9,7,8]`

Append



`[2,5,3,9,7,8,1]`

Sift up



`[1,5,2,9,7,8,3]`

The new value climbs only while it is smaller than its parent.

Sift up in Python

```python
def heappush(heap, value):
    heap.append(value)
    i = len(heap) - 1

    while i > 0:
        parent = (i - 1) // 2
        if heap[parent] <= heap[i]:
            break
        heap[parent], heap[i] = heap[i], heap[parent]
        i = parent
```

Sift up in Python

```python
def heappush(heap, value):
    heap.append(value)
    i = len(heap) - 1

    while i > 0:
        parent = (i - 1) // 2
        if heap[parent] <= heap[i]:
            break
        heap[parent], heap[i] = heap[i], heap[parent]
        i = parent
```

The loop moves upward through parent indexes, so it performs at most one swap per tree level.

Removing the minimum

Removing the minimum

1. Save the root: this is the minimum value.

Removing the minimum

1. Save the root: this is the minimum value.
2. Move the last element to the root.

Removing the minimum

1. Save the root: this is the minimum value.
2. Move the last element to the root.
3. Remove the last position.

Removing the minimum

1. Save the root: this is the minimum value.
2. Move the last element to the root.
3. Remove the last position.
4. Compare the root with its children.

Removing the minimum

1. Save the root: this is the minimum value.
2. Move the last element to the root.
3. Remove the last position.
4. Compare the root with its children.
5. Swap with the smaller child while the heap property is broken.

Removing the minimum

1. Save the root: this is the minimum value.
2. Move the last element to the root.
3. Remove the last position.
4. Compare the root with its children.
5. Swap with the smaller child while the heap property is broken.

Name of the repair
sift down

Removing the minimum

1. Save the root: this is the minimum value.
2. Move the last element to the root.
3. Remove the last position.
4. Compare the root with its children.
5. Swap with the smaller child while the heap property is broken.

Name of the repair

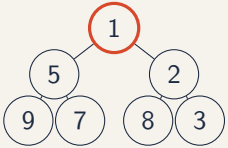
sift down

Removal is harder than insertion because the root disappears and the heap must choose the next best item.

Trace: popping the minimum

Trace: popping the minimum

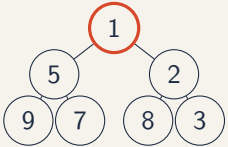
Start



`[1,5,2,9,7,8,3]`

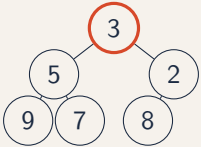
Trace: popping the minimum

Start



`[1,5,2,9,7,8,3]`

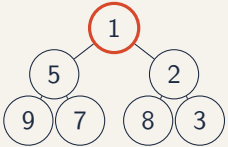
Move last to root



`[3,5,2,9,7,8]`

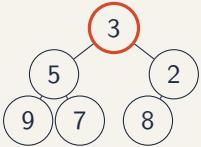
Trace: popping the minimum

Start



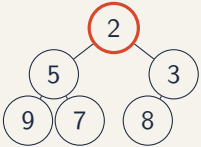
[1,5,2,9,7,8,3]

Move last to root



[3,5,2,9,7,8]

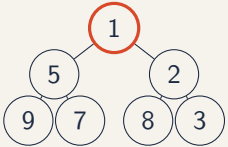
Sift down



[2,5,3,9,7,8]

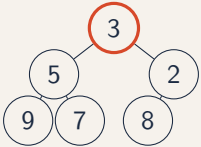
Trace: popping the minimum

Start



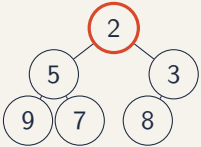
`[1,5,2,9,7,8,3]`

Move last to root



`[3,5,2,9,7,8]`

Sift down



`[2,5,3,9,7,8]`

The replacement value moves downward until both children are greater than or equal to it.

Sift down in Python

```python
def heappop(heap):
    result = heap[0]
    heap[0] = heap[-1]
    heap.pop()

    i = 0
    n = len(heap)
    while True:
        left = 2*i + 1
        right = 2*i + 2
        smallest = i

        if left < n and heap[left] < heap[smallest]:
            smallest = left
        if right < n and heap[right] < heap[smallest]:
            smallest = right
        if smallest == i:
            break

        heap[i], heap[smallest] = heap[smallest], heap[i]
        i = smallest

    return result
```

Why operations are logarithmic

Why operations are logarithmic

Complete shape

A heap with n elements has height:

$$O(\log n)$$

Why operations are logarithmic

Complete shape

A heap with n elements has height:

$$O(\log n)$$

Insertion

Sift up follows one path from leaf to root.

Why operations are logarithmic

Complete shape

A heap with n elements has height:

$$O(\log n)$$

Insertion

Sift up follows one path from leaf to root.

Removal

Sift down follows one path from root to leaf.

Why operations are logarithmic

Complete shape

A heap with n elements has height:

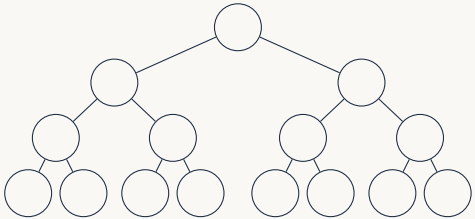
$$O(\log n)$$

Insertion

Sift up follows one path from leaf to root.

Removal

Sift down follows one path from root to leaf.



Why operations are logarithmic

Complete shape

A heap with n elements has height:

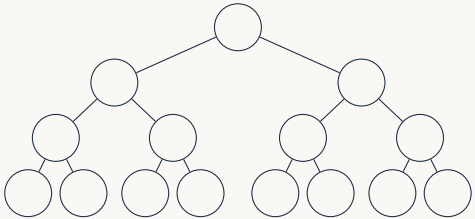
$$O(\log n)$$

Insertion

Sift up follows one path from leaf to root.

Removal

Sift down follows one path from root to leaf.



The heap never searches the whole tree during push or pop. It repairs one vertical path.

Python heapq

Python's heapq module

```python
import heapq

heap = []

heapq.heappush(heap, 5)
heapq.heappush(heap, 2)
heapq.heappush(heap, 8)

print(heap[0])              # 2
print(heapq.heappop(heap))  # 2
print(heap)                 # still a heap
```

Python's heapq module

```python
import heapq

heap = []

heapq.heappush(heap, 5)
heapq.heappush(heap, 2)
heapq.heappush(heap, 8)

print(heap[0])              # 2
print(heapq.heappop(heap))  # 2
print(heap)                 # still a heap
```

heapq works on normal Python lists. It does not create a special heap object.

The internal list is not sorted

The internal list is not sorted

A possible heap list

```
heap = [2, 5, 3, 9, 7, 8]
```

The internal list is not sorted

A possible heap list

```
heap = [2, 5, 3, 9, 7, 8]
```

This is a valid min-heap.

The internal list is not sorted

A possible heap list

```
heap = [2, 5, 3, 9, 7, 8]
```

This is a valid min-heap.

But it is not a sorted list.

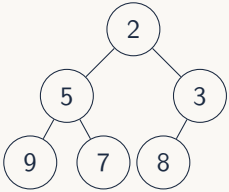
The internal list is not sorted

A possible heap list

```
heap = [2, 5, 3, 9, 7, 8]
```

This is a valid min-heap.

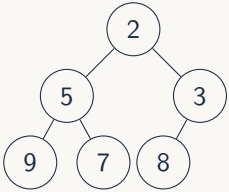
But it is not a sorted list.



The internal list is not sorted

A possible heap list

```
heap = [2, 5, 3, 9, 7, 8]
```



This is a valid min-heap.

But it is not a sorted list.

A heap keeps only enough order to make the next pop efficient.

Heapify

Heapify

Repeated insertion

```
heap = []
for x in data:
    heapq.heappush(heap, x)
```

Heapify

Repeated insertion

```
heap = []
for x in data:
    heapq.heappush(heap, x)
```

Cost: $O(n \log n)$

Heapify

Repeated insertion

```
heap = []
for x in data:
    heapq.heappush(heap, x)
```

Cost: $O(n \log n)$

In-place heap construction

```
heapq.heapify(data)
```

Heapify

Repeated insertion

```
heap = []
for x in data:
    heapq.heappush(heap, x)
```

Cost: $O(n \log n)$

In-place heap construction

```
heapq.heapify(data)
```

Cost: $O(n)$

Heapify

Repeated insertion

```
heap = []
for x in data:
    heapq.heappush(heap, x)
```

Cost: $O(n \log n)$

In-place heap construction

```
heapq.heapify(data)
```

Cost: $O(n)$

`heapify` transforms an existing list into a heap without first creating an empty heap.

Priority items with tuples

```python
import heapq

tasks = []

heapq.heappush(tasks, (3, "write report"))
heapq.heappush(tasks, (1, "fix server"))
heapq.heappush(tasks, (2, "answer email"))

while tasks:
    priority, task = heapq.heappop(tasks)
    print(task)
```

Priority items with tuples

```python
import heapq

tasks = []

heapq.heappush(tasks, (3, "write report"))
heapq.heappush(tasks, (1, "fix server"))
heapq.heappush(tasks, (2, "answer email"))

while tasks:
    priority, task = heapq.heappop(tasks)
    print(task)
```

Tuples are compared from left to right, so the priority value controls the heap order.

Output order

For this heap:

```
(3, "write report")
(1, "fix server")
(2, "answer email")
```

Output order

For this heap:

```
(3, "write report")
(1, "fix server")
(2, "answer email")
```

The output is:

Output order

For this heap:

```
(3, "write report")
(1, "fix server")
(2, "answer email")
```

The output is:

Result

```
fix server
answer email
write report
```

Output order

For this heap:

```
(3, "write report")
(1, "fix server")
(2, "answer email")
```

The output is:

Result

```
fix server
answer email
write report
```

In a min-heap, smaller numeric priority comes out first.

Tie-breaking equal priorities

Tie-breaking equal priorities

Risky pattern

```
heapq.heappush(tasks, (1, task_a))
heapq.heappush(tasks, (1, task_b))
```

Tie-breaking equal priorities

Risky pattern

```
heapq.heappush(tasks, (1, task_a))
heapq.heappush(tasks, (1, task_b))
```

If priorities tie, Python may compare the tasks themselves.

Tie-breaking equal priorities

Risky pattern

```
heapq.heappush(tasks, (1, task_a))
heapq.heappush(tasks, (1, task_b))
```

If priorities tie, Python may compare the tasks themselves.

Safer pattern

```
heapq.heappush(tasks,
               (priority, counter, task))
```

Tie-breaking equal priorities

Risky pattern

```
heapq.heappush(tasks, (1, task_a))
heapq.heappush(tasks, (1, task_b))
```

If priorities tie, Python may compare the tasks themselves.

Safer pattern

```
heapq.heappush(tasks,
               (priority, counter, task))
```

The counter breaks ties by insertion order.

Tie-breaking equal priorities

Risky pattern

```
heapq.heappush(tasks, (1, task_a))
heapq.heappush(tasks, (1, task_b))
```

If priorities tie, Python may compare the tasks themselves.

Safer pattern

```
heapq.heappush(tasks,
               (priority, counter, task))
```

The counter breaks ties by insertion order.

A common priority-queue entry is `(priority, counter, task)`.

When larger priority means more urgent

Python's `heapq` is min-heap oriented.

```python
import heapq

jobs = []

heapq.heappush(jobs, (-10, "emergency"))
heapq.heappush(jobs, (-2, "normal"))

priority, job = heapq.heappop(jobs)
print(job)  # emergency
```

When larger priority means more urgent

Python's `heapq` is min-heap oriented.

If larger numbers should come first, store negative priorities.

```python
import heapq

jobs = []

heapq.heappush(jobs, (-10, "emergency"))
heapq.heappush(jobs, (-2, "normal"))

priority, job = heapq.heappop(jobs)
print(job)  # emergency
```

When larger priority means more urgent

Python's `heapq` is min-heap oriented.

If larger numbers should come first, store negative priorities.

```python
import heapq

jobs = []

heapq.heappush(jobs, (-10, "emergency"))
heapq.heappush(jobs, (-2, "normal"))

priority, job = heapq.heappop(jobs)
print(job)  # emergency
```

Negating priorities turns a max-priority problem into a min-heap problem.

Priority Scheduling

Priority scheduling problem

We receive jobs with urgency levels.

Job	**Priority**
backup files	3
restart server	1
security alert	0
clean logs	5

Priority scheduling problem

We receive jobs with urgency levels.

Job	Priority
backup files	3
restart server	1
security alert	0
clean logs	5

Lower number means more urgent.

Priority scheduling problem

We receive jobs with urgency levels.

Job	Priority
backup files	3
restart server	1
security alert	0
clean logs	5

Lower number means more urgent.

The scheduler should repeatedly select the job with the smallest priority number.

Priority scheduler with heapq

```python
import heapq

jobs = [
    (3, "backup files"),
    (1, "restart server"),
    (0, "security alert"),
    (5, "clean logs"),
]

heapq.heapify(jobs)

while jobs:
    priority, job = heapq.heappop(jobs)
    print(job)
```

Priority scheduler with heapq

```python
import heapq

jobs = [
    (3, "backup files"),
    (1, "restart server"),
    (0, "security alert"),
    (5, "clean logs"),
]

heapq.heapify(jobs)

while jobs:
    priority, job = heapq.heappop(jobs)
    print(job)
```

The heap stores pending jobs. Each pop chooses the next job to execute.

Scheduling with stable tie-breaking

```python
import heapq

jobs = []
counter = 0

def add_job(priority, name):
    global counter
    heapq.heappush(jobs, (priority, counter, name))
    counter += 1

add_job(1, "restart server")
add_job(1, "answer admin email")
add_job(0, "security alert")

while jobs:
    priority, order, name = heapq.heappop(jobs)
    print(name)
```

Scheduling with stable tie-breaking

```python
import heapq

jobs = []
counter = 0

def add_job(priority, name):
    global counter
    heapq.heappush(jobs, (priority, counter, name))
    counter += 1

add_job(1, "restart server")
add_job(1, "answer admin email")
add_job(0, "security alert")

while jobs:
    priority, order, name = heapq.heappop(jobs)
    print(name)
```

When priorities are equal, the counter makes the scheduler behave predictably.

Where priority queues appear

Where priority queues appear

Scheduling

Operating systems, task managers, printers, emergency systems.

Where priority queues appear

Scheduling

Operating systems, task managers, printers, emergency systems.

Simulation

Process the next event by earliest timestamp.

Where priority queues appear

Scheduling

Operating systems, task managers, printers, emergency systems.

Simulation

Process the next event by earliest timestamp.

Algorithms

Dijkstra's algorithm and Prim's algorithm need repeated extraction of the best candidate.

Where priority queues appear

Scheduling

Operating systems, task managers, printers, emergency systems.

Algorithms

Dijkstra's algorithm and Prim's algorithm need repeated extraction of the best candidate.

Simulation

Process the next event by earliest timestamp.

Data analysis

Keep the top k items without sorting everything.

Where priority queues appear

Scheduling

Operating systems, task managers, printers, emergency systems.

Algorithms

Dijkstra's algorithm and Prim's algorithm need repeated extraction of the best candidate.

Simulation

Process the next event by earliest timestamp.

Data analysis

Keep the top k items without sorting everything.

Priority queues are useful whenever *next best* matters more than full sorting.

Complexity and Limits

Complexity summary

Operation	Cost	Reason
`heap[0]` / peek	$O(1)$	best item is at the root
`heappush`	$O(\log n)$	sift up one path
`heappop`	$O(\log n)$	sift down one path
`heapify`	$O(n)$	build heap in place
Pop all items	$O(n \log n)$	n removals

Complexity summary

Operation	Cost	Reason
`heap[0]` / peek	$O(1)$	best item is at the root
`heappush`	$O(\log n)$	sift up one path
`heappop`	$O(\log n)$	sift down one path
`heapify`	$O(n)$	build heap in place
Pop all items	$O(n \log n)$	n removals

A heap gives fast access to the next best item, not fast access to every item.

What heaps are bad at

What heaps are bad at

- searching for a specific value;

What heaps are bad at

- searching for a specific value;
- printing all values in sorted order without popping;

What heaps are bad at

- searching for a specific value;
- printing all values in sorted order without popping;
- removing an arbitrary internal value;

What heaps are bad at

- searching for a specific value;
- printing all values in sorted order without popping;
- removing an arbitrary internal value;
- updating a priority already inside the heap;

What heaps are bad at

- searching for a specific value;
- printing all values in sorted order without popping;
- removing an arbitrary internal value;
- updating a priority already inside the heap;
- answering general order queries such as “what is the second smallest?” without extra reasoning.

What heaps are bad at

- searching for a specific value;
- printing all values in sorted order without popping;
- removing an arbitrary internal value;
- updating a priority already inside the heap;
- answering general order queries such as "what is the second smallest?" without extra reasoning.

A heap is specialized. It is not a replacement for BSTs, sorted arrays, or hash tables.

Choosing the right structure

Structure	Rule	Best for
Queue	first in, first out	arrival order
Stack	last in, first out	undo, backtracking
BST	left $<$ node $<$ right	searching by value
AVL tree	BST $+$ balance	reliable search
Heap	parent better than children	repeated best-item removal
Priority queue	ADT: pop best item	scheduling and events

Choosing the right structure

Structure	Rule	Best for
Queue	first in, first out	arrival order
Stack	last in, first out	undo, backtracking
BST	left $<$ node $<$ right	searching by value
AVL tree	BST $+$ balance	reliable search
Heap	parent better than children	repeated best-item removal
Priority queue	ADT: pop best item	scheduling and events

The right data structure depends on the question you need to answer quickly.

Wrap-up

Takeaways

Takeaways

- A priority queue removes items by priority, not arrival order.

Takeaways

- A priority queue removes items by priority, not arrival order.
- A binary heap is a complete binary tree with a heap property.

Takeaways

- A priority queue removes items by priority, not arrival order.
- A binary heap is a complete binary tree with a heap property.
- A heap is not a BST: it has a weaker ordering rule.

Takeaways

- A priority queue removes items by priority, not arrival order.
- A binary heap is a complete binary tree with a heap property.
- A heap is not a BST: it has a weaker ordering rule.
- The complete shape allows compact list storage.

Takeaways

- A priority queue removes items by priority, not arrival order.
- A binary heap is a complete binary tree with a heap property.
- A heap is not a BST: it has a weaker ordering rule.
- The complete shape allows compact list storage.
- Insertion uses sift up; removal uses sift down.

Takeaways

- A priority queue removes items by priority, not arrival order.
- A binary heap is a complete binary tree with a heap property.
- A heap is not a BST: it has a weaker ordering rule.
- The complete shape allows compact list storage.
- Insertion uses sift up; removal uses sift down.
- Python's `heapq` implements a min-heap on top of a normal list.

Takeaways

- A priority queue removes items by priority, not arrival order.
- A binary heap is a complete binary tree with a heap property.
- A heap is not a BST: it has a weaker ordering rule.
- The complete shape allows compact list storage.
- Insertion uses sift up; removal uses sift down.
- Python's `heapq` implements a min-heap on top of a normal list.
- Priority scheduling is a direct application of heaps.

Takeaways

- A priority queue removes items by priority, not arrival order.
- A binary heap is a complete binary tree with a heap property.
- A heap is not a BST: it has a weaker ordering rule.
- The complete shape allows compact list storage.
- Insertion uses sift up; removal uses sift down.
- Python's `heapq` implements a min-heap on top of a normal list.
- Priority scheduling is a direct application of heaps.

A heap does not keep everything sorted; it keeps just enough order to find the next item efficiently.

Preview: graph algorithms

Priority queues will return later in graph algorithms.

Preview: graph algorithms

Priority queues will return later in graph algorithms.

Example

Dijkstra's algorithm repeatedly selects the unvisited node with the smallest current distance.

Preview: graph algorithms

Priority queues will return later in graph algorithms.

Example

Dijkstra's algorithm repeatedly selects the unvisited node with the smallest current distance.

Remark 9.3. Connection

That selection step is exactly a priority-queue operation.

Preview: graph algorithms

Priority queues will return later in graph algorithms.

Example

Dijkstra's algorithm repeatedly selects the unvisited node with the smallest current distance.

Remark 9.4. Connection

That selection step is exactly a priority-queue operation.

Heaps are small structures with large consequences: scheduling, simulations, shortest paths, and optimization all rely on them.